

Description of the Learning Algorithm

Algorithm Overview

This work presents a supervised regression algorithm that utilizes a Graph Neural Network (GNN) to learn the forward kinematics (FK) mapping for robotic arms. The model is designed to predict the end-effector pose from a set of joint angles, leveraging the inherent graph structure of a robot manipulator. A key feature is the use of transfer learning, where knowledge acquired from a simpler robot model is fine-tuned for a more complex one.

1 Problem Definition

The algorithm solves the forward kinematics function:

$$f_{\theta} : \mathbb{R}^6 \rightarrow \mathbb{R}^m$$

where the input is a vector of six joint angles, $\mathbf{q} = [q_1, q_2, q_3, q_4, q_5, q_6]$, and the output dimension m depends on the target robot:

- For the IRB120 robot: $m = 3$, outputting Cartesian coordinates $[x, y, z]$.
- For the IRB2400 robot: $m = 6$, outputting full pose $[x, y, z, \text{roll}, \text{pitch}, \text{yaw}]$.

The goal is to learn the optimal parameters θ of the model that minimize the prediction error between the predicted and the true end-effector pose.

2 Graph-Based Representation

Instead of treating the joint angles as an unstructured vector, the robot is represented as a graph:

- **Nodes:** Represent the six joints.
- **Edges:** Encode the physical connections between consecutive joints in the kinematic chain, with added self-loops.

This structure is encoded by the normalized adjacency matrix $\hat{\mathbf{A}}$:

$$\hat{\mathbf{A}} = \mathbf{D}^{-\frac{1}{2}} \mathbf{A} \mathbf{D}^{-\frac{1}{2}}$$

where $\mathbf{D}$ is the degree matrix. This graph guides the message-passing mechanism of the GNN, allowing it to learn the spatial relationships and interactions between joints.

3 Model Architecture

The GNN architecture consists of the following components:

1. **GNN Backbone:** Three deep Graph Convolutional Network (GCN) blocks. Each block performs the following operation:

$$\mathbf{H}^{(l+1)} = \text{LayerNorm} \left(\sigma \left(\hat{\mathbf{A}} \mathbf{H}^{(l)} \mathbf{W}^{(l)} + \mathbf{b}^{(l)} \right) + \mathbf{H}^{(l)} \right)$$

where σ is the ReLU activation function. Residual connections and layer normalization are used to improve training stability and enable deeper networks.

2. **Global Readout:** A graph-level embedding is created by concatenating the outputs of global average pooling and global max pooling operations across all nodes.
3. **MLP Head:** A series of fully connected layers that map the graph-level embedding to the final output dimension (3 or 6).

4 Learning Process

The model is trained in a supervised manner:

- **Loss Function:** Mean Squared Error (MSE) is used to optimize the model parameters:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N \|\hat{\mathbf{y}}_i - \mathbf{y}_i\|^2$$

where $\hat{\mathbf{y}}_i$ is the predicted pose and $\mathbf{y}_i$ is the true pose for the i -th sample.

- **Optimizer:** The Adam optimizer is employed with an initial learning rate of 1×10^{-4} , which is adaptively reduced using a `ReduceLROnPlateau` scheduler.
- **Training Strategy:** The training follows a two-stage transfer learning pipeline:
 1. **Pretraining:** The model is trained on data from the IRB120 robot (predicting x, y, z).
 2. **Fine-tuning:** The model is then initialized with the pretrained weights and fine-tuned on the IRB2400 dataset to predict the full 6D pose ($x, y, z, \text{roll}, \text{pitch}, \text{yaw}$).

5 Transfer Learning Mechanism

The algorithm employs transfer learning to leverage knowledge from a source domain (IRB120) to improve learning in a target domain (IRB2400):

- The lower GCN layers learn general, transferable patterns of joint interactions and kinematics.
- Only the final regression head is modified to accommodate the different output size of the new robot.
- Fine-tuning adjusts the general patterns to the specific morphology of the new robot, leading to faster convergence and better performance.

6 Algorithm Summary

- **Input:** Joint angle vector $\mathbf{q}$.
- **Graph Encoding:** Construction of the normalized adjacency matrix $\hat{\mathbf{A}}$ for a 6-node chain graph.
- **Forward Pass:** Processing through the GNN backbone, global pooling, and an MLP head.
- **Output:** Prediction of the end-effector pose.
- **Optimization:** Minimization of the MSE loss via gradient descent.
- **Learning Paradigm:** Supervised regression with a GNN model.
- **Key Feature:** Domain-specific transfer learning via fine-tuning across different robotic morphologies.

In summary, this algorithm trains a graph neural network to approximate the forward kinematics function of robotic arms, utilizing a structured representation and transfer learning to efficiently scale from simpler to more complex robots.